SISTEMAS OPERATIVOS Y REDES II (A0533)

LICENCIATURA EN SISTEMAS

2026 - 1° SEMESTRE

COM 01 - UNGS - INSTITUTO DE INDUSTRIA

INFORME - GRUPO 6

TRABAJO PRÁCTICO 1

SISTEMA DE ARCHIVOS FAT

ALUMNOS:

- ABREGU, PABLO
- BENITEZ, GONZALO
- CABALLERO, CARLOS
- GALLARDO, JUAN
- HERRERA, NAHUEL
- PUCHETA, EMMANUEL

FECHA DE ENTREGA 15/04/26

ÍNDICE

1. INTRODUCCIÓN 3
1.1. Contexto y objetivo del trabajo 3
1.2. Distribución de tareas entre integrantes 3
2. ENTORNO DE TRABAJO 4
2.1. Sistema Operativo 4
2.2. Recursos de la VM 4
2.3. Herramientas utilizadas 4
2.4. Estado inicial del sistema 4
3. DESARROLLO 5
3.1. Montaje del Sistema de Archivos 5
3.2. Cargando el MBR 5
a. Mostrando el MBR con el Hex Editor 5
b. Leer los datos del MBR utilizando código C 7
c. Verificar si la primera partición es booteable 7
d. Mostrar información de la primera partición 8
3.3. Cargando la Tabla de Archivos 8
a. Mostrar archivos en el filesystem 8
b. Crear y borrar un archivo 9
c. Mostrar archivos borrados 10
d. Recupero de archivos 11
3.4. Leyendo Archivos 12
a. Crear un archivo y mostrarlo 12
b. Mostrar el contenido de un archivo no borrado 14
c. Recuperar un archivo borrado 15
4. CONCLUSIONES 18
5. REFERENCIAS 19
6. ANEXOS 20
6. DECLARACIÓN DE AUTORÍA 20

1. INTRODUCCIÓN

1.1. Contexto y objetivo del trabajo

El objetivo de este trabajo es explorar y analizar el sistema de archivos FAT12, utilizando una imagen de disco (test.img). Durante el trabajo se buscará demostrar y analizar el MBR (Master Boot Sector), entender la estructura del Boot Sector de FAT12, leer y manipular el Directorio Raíz, recuperar archivos borrados y analizar la gestión de archivos en FAT12.

1.2. Distribución de tareas entre integrantes

INTEGRANTE	TAREA A CARGO
ABREGU, PABLO	Leyendo archivos - Punto 4
BENITEZ, GONZALO	Organización del informe - Punto 1
CABALLERO, CARLOS	Leer root - Punto 3
GALLARDO, JUAN	Leer MBR
HERRERA, NAHUEL	Organización del informe
PUCHETA, EMMANUEL	Organización del informe

2. ENTORNO DE TRABAJO

2.1. Sistema Operativo

Se trabajó sobre una Máquina Virtual (VM) con Sistema Operativo Lubuntu, una distribución basada en Ubuntu.

2.2. Recursos de la VM

La máquina virtual utilizada para el desarrollo de la práctica cuenta con las siguientes especificaciones:

- Memoria RAM: 8 GB
- Procesador: 6 CPU
- Almacenamiento: 150 GB

2.3. Herramientas utilizadas

Para la realización del trabajo práctico se utilizaron las siguientes herramientas:

- GHex: editor hexadecimal utilizado para inspeccionar la imagen de disco *test.img*.
- GCC: compilador de C utilizado para compilar y ejecutar los programas de la práctica.
- Terminal de Linux (QTerminal): utilizada para montar la imagen, navegar por el sistema y ejecutar comandos.
- Código en C: read_mbr.c, read_boot.c, read_root.c

2.4. Estado inicial del sistema

Se contó con una imagen de disco *test.img*, correspondiente a un sistema de archivos FAT12, sobre la cual se realizan tareas de análisis.

3. DESARROLLO

3.1. Montaje del Sistema de Archivos

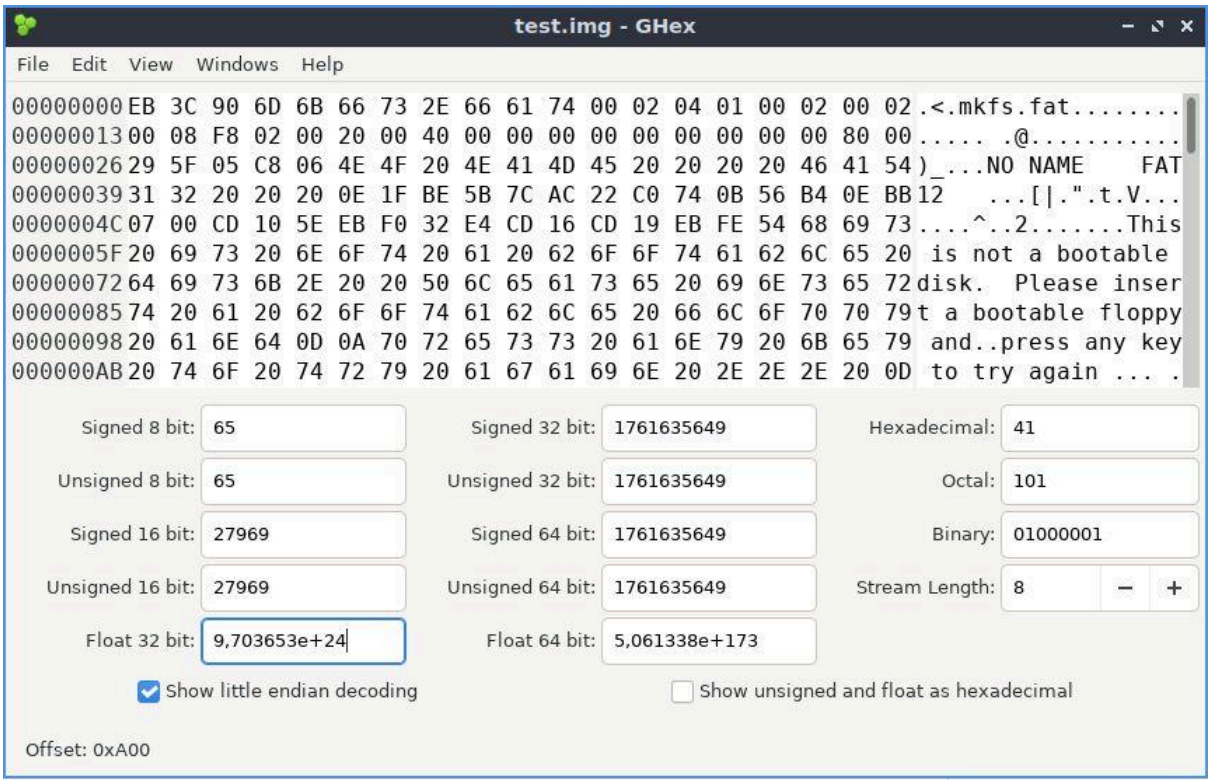
¿Para qué se ha puesto umask=000 al montar el sistema de archivos? Explicar el propósito de este parámetro.

Cuando se crea un archivo o directorio, el sistema le asigna permisos automáticamente. Dichos permisos surgen de los permisos máximos posibles y la umask del usuario. El parámetro umask=000 se utiliza al montar el sistema para no restringir ningún permiso sobre los archivos.

De esta manera, todos los usuarios pueden leer y escribir archivos, y en el caso de los directorios también acceder y modificarlos, evitando problemas de permisos.

3.2. Cargando el MBR

a. Mostrando el MBR con el Hex Editor

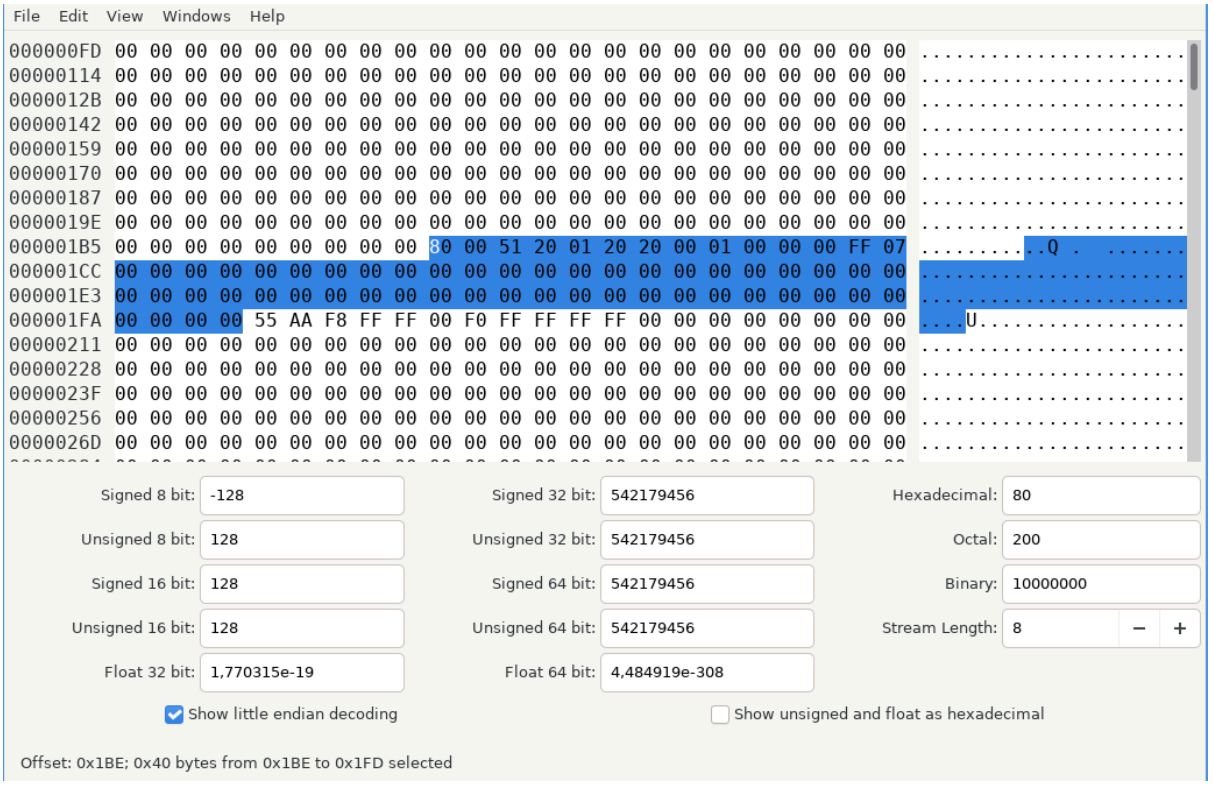


En esta imagen se observan los bytes de la imagen de disco (test.img) en el editor hexadecimal **GHex**, donde se observa parte del sector 0 reservado por el sistema de archivos,

que corresponde al MBR (Master Boot Sector). El MBR (Master Boot Record) ocupa 512 bytes, los cuales se distribuyen de la siguiente manera:

- **Código de arranque:** 446 bytes
- **Entradas de partición:** 16 bytes por cada una de las cuatro entradas (Partición 1, 2, 3 y 4).
- **Firma del MBR (0x55AA):** 2 bytes

Esta estructura se alinea con lo visto en los apuntes.



En esta segunda imagen, se observan los bytes finales del código de arranque del MBR y los 64 bytes correspondientes a las entradas de las 4 particiones que admite FAT12 (16 bytes x 4 particiones) marcados en la imagen. Como se puede ver, sólo hay datos en la Entrada de partición 1, lo que significa que este disco se divide en una sola partición ya que las demás entradas de particiones contienen ceros.

Además, en esta imagen también se observa la firma del MBR en el offset 0x1FE-0x1FF.

b. Leer los datos del MBR utilizando código C

Datos de la entrada de partición 1:

- **80**: Partición activa (*0x80*)
- **00 51 20**: Dirección CHS de inicio
- **01**: Tipo de partición (*0x01=FAT12*)
- **20 20 00**: Dirección CHS de fin
- **01 00 00 00**: LBA de inicio de partición (*1*)
- **FF 07 00 00**: Tamaño de la partición en sectores (*2047*)

Para observar esta información de manera más clara, crearemos un programa en c llamado *read_mbr.c* para leer estos datos:

```
alumno@alumno-virtualbox:~/FAT12$ ./read_mbr
Partition entry 0: First byte 80
  Comienzo de partición en CHS: 00:02:00
  Partition type 0x01
  Fin de partición en CHS: 00:20:20
  Dirección LBA relativa 0x00000001, de tamaño en sectores 2047
Partition entry 1: First byte 00
  Comienzo de partición en CHS: 00:00:00
  Partition type 0x00
  Fin de partición en CHS: 00:00:00
  Dirección LBA relativa 0x00000000, de tamaño en sectores 0
Partition entry 2: First byte 00
  Comienzo de partición en CHS: 00:00:00
  Partition type 0x00
  Fin de partición en CHS: 00:00:00
  Dirección LBA relativa 0x00000000, de tamaño en sectores 0
Partition entry 3: First byte 00
  Comienzo de partición en CHS: 00:00:00
  Partition type 0x00
  Fin de partición en CHS: 00:00:00
  Dirección LBA relativa 0x00000000, de tamaño en sectores 0
alumno@alumno-virtualbox:~/FAT12$
```

c. Verificar si la primera partición es booteable

La primera partición es booteable ya que el valor del offset 0x00 es 0x80, lo que determina que está activa.

d. Mostrar información de la primera partición

```
alumno@alumno-virtualbox:~/FAT12$ ./read_mbr
Partition entry 0: First byte 80
  Comienzo de partición en CHS: 00:02:00
  Partition type 0x01
  Fin de partición en CHS: 00:20:20
  Dirección LBA relativa 0x00000001, de tamaño en sectores 2047
alumno@alumno-virtualbox:~/FAT12$
```

3.3. Cargando la Tabla de Archivos

a. Mostrar archivos en el filesystem

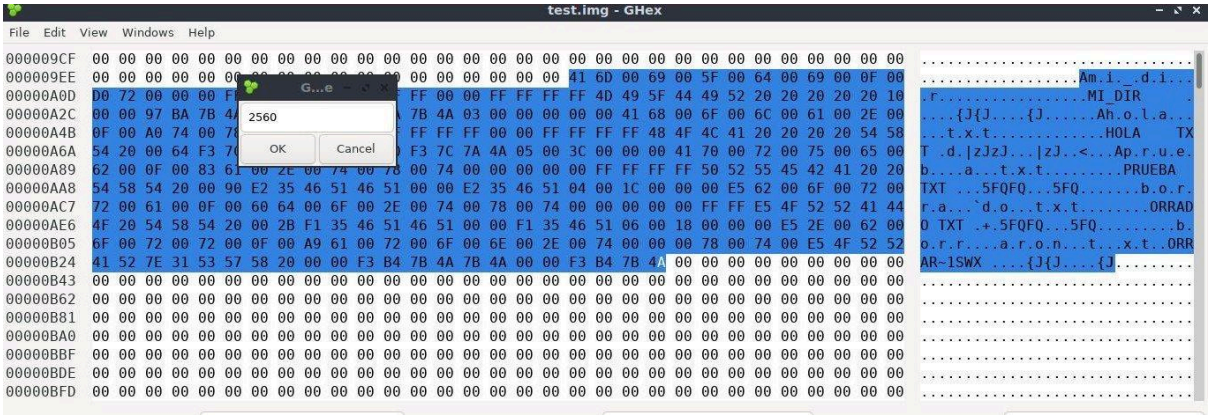
Para mostrar cuantos y cuales archivos tiene el filesystem primero debemos determinar en cual byte comienza el directorio raíz, lo cual implica analizar el boot sector. Para esto debemos observar:

- El número de sectores reservados: esto se encuentra en los bytes 14 y 15 del boot sector, en nuestro caso 1.
- El número de sectores por FAT: se encuentra en los bytes 22 y 23, en nuestro caso es 2.

En FAT12, hay dos copias de la tabla, por lo que el número total de sectores ocupados para almacenar la tabla FAT es igual al número de copias de la tabla FAT multiplicado por el número de sectores por FAT, finalmente es 2*2=4.

Ya teniendo el número de sectores reservados y el número de sectores por FAT, calculamos el número del sector que contiene el directorio raíz. Este valor se obtiene sumando los sectores reservados y los sectores utilizados por FAT. En este caso 1+4=5. Por lo tanto el directorio raíz comienza en el sector 5.

Finalmente calculamos la dirección del primer byte del directorio raíz, para esto multiplicamos el número del sector por el tamaño de cada sector (512 bytes) 5*512 = 2560 bytes. Este valor es el byte donde comienza el directorio raíz



Cada entrada del directorio raíz ocupa 32 bytes. Para identificar cuales archivos son válidos revisaremos el primer byte de cada entrada del directorio. Una entrada es válida si su primer byte no es 0xE5 ni tampoco 0x00 ya que indica el final del directorio.

En la imagen montada tenemos 6 entradas que son válidas ya que no comienzan con 0xE5 y 4 entradas que no son válidas porque comienzan con 0xE5.

Luego de corregir read root y ejecutarlo por la terminal obtenemos lo siguiente:

```
alumno@alumno-virtualbox: ~/Desktop/UNGS/Sor2-2026-TP1
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ gcc read_root.c -o read_root
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ ./read_root
Encontrada particion FAT12 0
En  0x200, sector size 512, FAT size 2 sectors, 2 FATs

Root dir_entries 512
Directorio: [MI_DIR  .   ]

Archivo: [HOLA    .TXT] su contenido es: Hola, bienvenidos !
Pueden ahora tratar de leerme desde c !

Archivo: [PRUEBA  .TXT] su contenido es: este es un archivo de prueba
Archivo borrado: [?b.]
Archivo borrado: [?ORRADO T.TXT]
Archivo borrado: [?..]
Archivo borrado: [?ORRAR~1S.SWX]

Leido Root directory, ahora en 0x4A00
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$
```

b. Crear y borrar un archivo

Para lograr esto ejecutaremos los siguientes pasos:

1 - Primero montamos la imagen en el sistema.

sudo mount test.img /mnt -o loop,umask=000

2 - Creamos el archivo SOR2TP1.txt utilizando el comando touch.

touch /mnt/SOR2TP1.txt

3 - Borramos el archivo con el comando rm

rm /mnt/SOR2TP1.txt

```
alumno@alumno-virtualbox: /mnt
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ sudo mount test.img /mnt -o loop,umask=000
[sudo] contraseña para alumno:
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ touch /mnt/SOR2TP1.txt
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ ls /mnt
hola.txt  mi_dir  prueba.txt  SOR2TP1.txt
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ rm SOR2TP1.txt
rm: no se puede borrar 'SOR2TP1.txt': No existe el archivo o el directorio
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ rm /mnt/SOR2TP1.txt
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ ls /mnt
hola.txt  mi_dir  prueba.txt
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ cd /mnt
alumno@alumno-virtualbox:/mnt$
```

4 - Ejecutamos de nuevo read root para ver el archivo borrado.

```
Archivo  Acciones  Editar  Vista  Ayuda
alumno@alumno-virtualbox: ~/Desktop/UNGS/Sor2-2026-TP1
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ ./read_root
Encontrada particion FAT12 0
En  0x200, sector size 512, FAT size 2 sectors, 2 FATs

Root dir_entries 512
Directorio: [MI_DIR  .   ]

Archivo: [HOLA    .TXT] su contenido es: Hola, bienvenidos !
Pueden ahora tratar de leerme desde c !

Archivo: [PRUEBA  .TXT] su contenido es: este es un archivo de prueba
Archivo borrado: [?S.]
Archivo borrado: [?OR2TP1 T.TXT]
Archivo borrado: [?..]
Archivo borrado: [?ORRAR~1S.SWX]

Leido Root directory, ahora en 0x4A00
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$
```

c. Mostrar archivos borrados

Para visualizar el archivo borrado en GHEX primero debemos buscar el directorio raíz, retomamos los visto en el punto 3.a donde encontramos que era el byte 2560.

Una vez en el, debemos mirar las entradas, cada entrada tiene 32 bytes y fijarnos si el primer byte comienza con 0xE5, esto indica que el archivo ha sido borrado. Para saber si el archivo borrado es el nuestro, miramos la tabla ascii si corresponde al nombre que se le había puesto.

```
File  Edit  View  Windows  Help
00000AA0 50 52 55 45 42 41 20 20 54 58 54 20 00 90 E2 35 PRUEBA  TXT ...5
00000AB0 46 51 46 51 00 00 E2 35 46 51 04 00 1C 00 00 00 FQFQ...5FQ......
00000AC0 E5 53 00 4F 00 52 00 32 00 54 00 0F 00 AF 50 00 .S.O.R.2.T....P.
00000AD0 31 00 2E 00 74 00 78 00 74 00 00 00 00 00 FF FF 1...t.x.t.......
00000AE0 E5 4F 52 32 54 50 31 20 54 58 54 20 00 75 F1 38 .OR2TP1 TXT .u.8
00000AF0 88 5C 88 5C 00 00 F1 38 88 5C 00 00 00 00 00 00 .\.\...8.\......
00000B00 E5 2E 00 62 00 6F 00 72 00 72 00 0F 00 A9 61 00 ...b.o.r.r....a.
00000B10 72 00 6F 00 6E 00 2E 00 74 00 00 00 78 00 74 00 r.o.n...t...x.t.
00000B20 E5 4F 52 52 41 52 7E 31 53 57 58 20 00 00 F3 B4 .ORRAR~1SWX ....
00000B30 7B 4A 7B 4A 00 00 F3 B4 7B 4A 00 00 00 00 00 00 {J{J....{J......
```

d. Recupero de archivos

La recuperación de archivos en FAT12 se basa en la estructura de la tabla FAT y las entradas del directorio. Cuando un archivo es borrado solo se borra la referencia a memoria en el directorio y se marca como 0xE5 indicando como espacio libre en la FAT, pero los datos siguen en el disco. La tabla FAT guarda la ubicación de los clusters de datos del archivo.

Para recuperar un archivo, se buscan los cluster del archivo en la FAT. Si los clusters no han sido sobreescritos la recuperación es posible. Sin embargo si han sido sobreescritas se vuelve imposible. Para la recuperación del archivo, se localiza la entrada de directorio, se identifica el primer cluster del archivo y se sigue la cadena de cluster en la tabla FAT. Luego se extraen los datos de esos clusters. Por último se restaura el archivo.

Para esto creamos el programa para la recuperación de archivos: *recuperar_archivo.c*

Luego de ejecutarlo obtenemos la siguiente salida.

```
alumno@alumno-virtualbox: ~/Desktop/UNGS/Sor2-2026-TP1
Archivo  Acciones  Editar  Vista  Ayuda
alumno@alumno-virtualbox: ~/Desktop/UNGS/Sor2-2026-TP1
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ ./recuperar_archivo
1. Archivo recuperable: [?ORRADO T.TXT] Tamaño: 24 bytes, Clusters: 2

Se encontraron 1 archivos recuperables.
Ingrese el número del archivo a recuperar (0 para salir): 1
Ingrese el primer carácter del nombre original: B
Archivo recuperado: [BORRADO .TXT]
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$
```

Al ejecutar el programa de recuperar archivos, se ve que solamente 1 archivo de 4 son recuperables, en este caso “?ORRADO.TXT”. El programa pide ingresar el número del archivo a recuperar, en este caso ingresamos 1 ya que hay uno solo. Y luego nos pide

ingresar el primer carácter para reemplazarlo y poder recuperar el archivo además de que será el primer carácter del nombre del archivo recuperado. En este caso ponemos la “B”, ya que deducimos que el archivo era “BORRADO.TXT”.

Después al leer los archivos que contiene la imagen, vemos cómo se recuperó el archivo, ahora con nombre “BORRADO.TXT” por la “B” que ingresamos anteriormente y se muestra su contenido dentro.

```
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$ ./read_root
Encontrada particion FAT12 0
En  0x200, sector size 512, FAT size 2 sectors, 2 FATs

Root dir_entries 512
Directorio: [MI_DIR  .   ]

Archivo: [HOLA    .TXT] su contenido es: Hola, bienvenidos !
Pueden ahora tratar de leerme desde c !

Archivo: [PRUEBA  .TXT] su contenido es: este es un archivo de prueba
Archivo borrado: [?b.]
Archivo: [BORRADO .TXT] su contenido es: este archivo fue borrado
Archivo borrado: [?..]
Archivo borrado: [?ORRAR~1S.SWX]

Leido Root directory, ahora en 0x4A00
alumno@alumno-virtualbox:~/Desktop/UNGS/Sor2-2026-TP1$
```

3.4. Leyendo Archivos

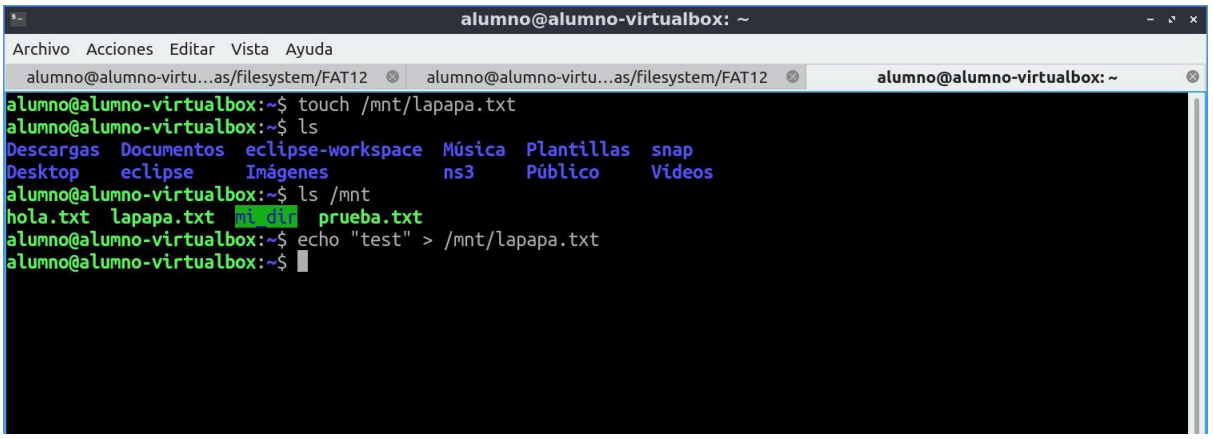
a. Crear un archivo y mostrarlo

Primero, montamos el filesystem test.img usando el siguiente comando:

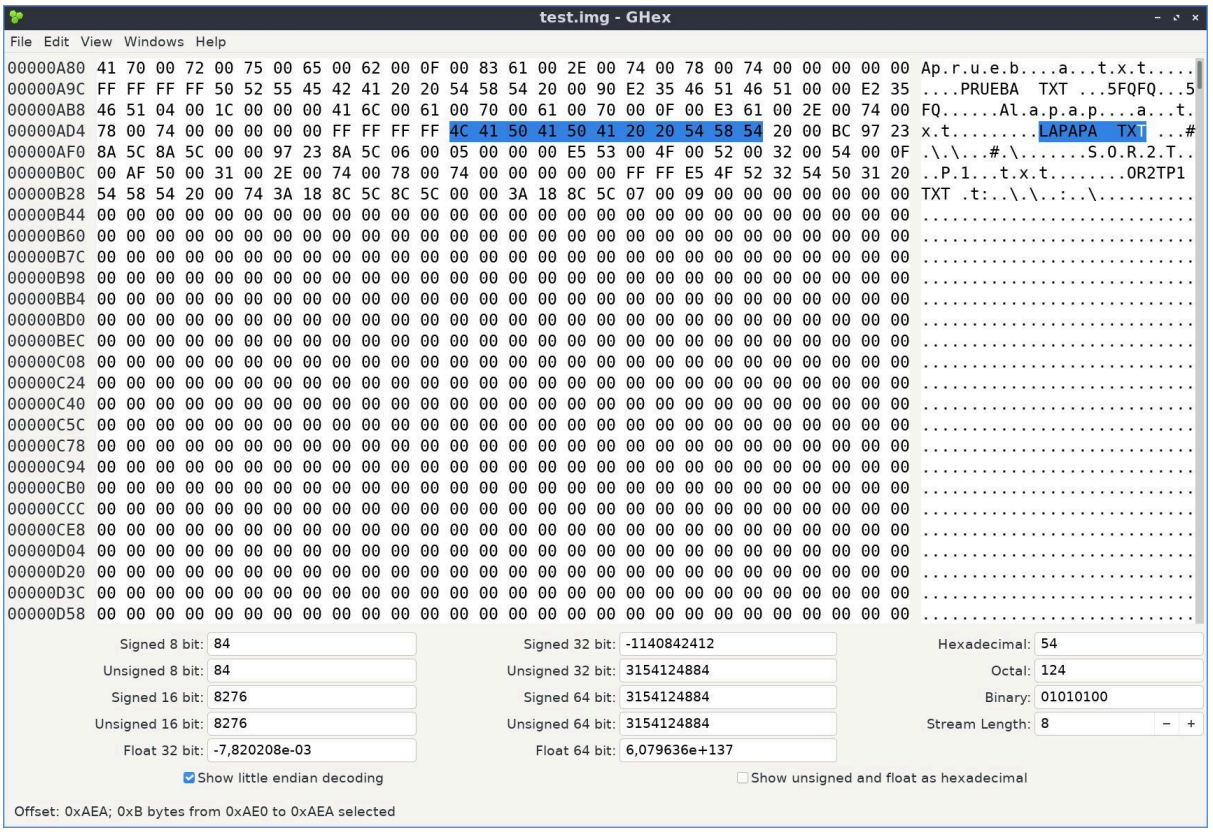
```
sudo mount test.img /mnt -o loop,umask=000
```

```
alumno@alumno-virtualbox: ~/Descargas/filesystem/FAT12
Archivo  Acciones  Editar  Vista  Ayuda
alumno@alumno-virtu...as/filesystem/FAT12
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ pwd
/home/alumno/Descargas/filesystem/FAT12
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ ls
Makefile  read_boot.c  read_mbr.c  read_root.c  test.img  TP1_FileSystem_2025_2S.pdf
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ sudo mount test.img /mnt -o loop,umask=000
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ ls /mnt
hola.txt  mi_dir  prueba.txt
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$
```

Luego, creamos un archivo llamado lapapa.txt en la carpeta raíz (/) usando el comando touch /mnt/lapapa.txt, luego listamos con ls /mnt y vemos que se encuentra el archivo.



Buscamos el archivo en GHex, donde el nombre aparece como LAPAPA TXT. Esto se debe a que el sistema de archivos FAT12 utiliza un formato 8.3, en el cual el nombre del archivo se almacena utilizando 8 bytes y la extensión 3 bytes, sin el punto separador. Esto corresponde a los valores hexadecimales 4C 41 50 41 50 41 20 20 54 58 54.



Por último, buscamos el archivo con el programa *read_root.c* y vemos que aparece listado como Archivo: [LAPAPA .TXT].

```
alumno@alumno-virtualbox: ~/Descargas/filesystem/FAT12
Archivo  Acciones  Editar  Vista  Ayuda
alumno@alumno-virtu...as/filesystem/FAT12
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ gcc read_root.c -o read_root
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ ./read_root
Encontrada particion FAT12 0
En  0x200, sector size 512, FAT size 2 sectors, 2 FATs

Root dir_entries 512
Directorio: [MI_DIR  .   ]

Archivo: [HOLA    .TXT]
Archivo: [PRUEBA  .TXT]
Archivo: [LAPAPA  .TXT]
Archivo borrado: [?S.]
Archivo borrado: [?OR2TP1 T.TXT]

Leido Root directory, ahora en 0x4A00
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$
```

b. Mostrar el contenido de un archivo no borrado

Primero, mostramos el contenido del archivo lapapa.txt usando GHex. Se observan los bytes 74 65 73 74, que corresponden al texto test agregado al archivo en el ítem anterior.

```
test.img - GHex
File  Edit  View  Windows  Help
00006804 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006820 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
0000683C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006858 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006874 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006890 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
000068AC 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
000068C8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
000068E4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006900 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
0000691C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006938 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006954 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006970 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
0000698C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
000069A8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
000069C4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
000069E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
000069FC 00 00 00 00 74 65 73 74 0A 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ....test......................
00006A18 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006A34 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006A50 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006A6C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006A88 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006AA4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006AC0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................
00006ADC 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ..............................

Signed 8 bit: 116          Signed 32 bit: 1953719668          Hexadecimal: 74
Unsigned 8 bit: 116        Unsigned 32 bit: 1953719668        Octal: 164
Signed 16 bit: 25972       Signed 64 bit: 1953719668          Binary: 01110100
Unsigned 16 bit: 25972     Unsigned 64 bit: 1953719668        Stream Length: 8
Float 32 bit: 7,713537e+31 Float 64 bit: 2,218522e-313
☑ Show little endian decoding        ☐ Show unsigned and float as hexadecimal

Offset: 0x6A00; 0x4 bytes from 0x6A00 to 0x6A03 selected
```

Luego, modificamos el programa *read_root.c* para mostrar el contenido de los archivos no borrados.

```
alumno@alumno-virtualbox: ~/Descargas/filesystem/FAT12
Archivo  Acciones  Editar  Vista  Ayuda
alumno@alumno-virtu...as/filesystem/FAT12
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ gcc read_root.c -o read_root
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ ./read_root
Encontrada particion FAT12 0
En  0x200, sector size 512, FAT size 2 sectors, 2 FATs

Root dir_entries 512
Directorio: [MI_DIR  .   ]

Archivo: [HOLA    .TXT] su contenido es:
Hola, bienvenidos !
Pueden ahora tratar de leerme desde c !


Archivo: [PRUEBA  .TXT] su contenido es:
este es un archivo de prueba

Archivo: [LAPAPA  .TXT] su contenido es:
test


Archivo borrado: [?S.]
Archivo borrado: [?OR2TP1 T.TXT]

Leido Root directory, ahora en 0x4A00
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$
```

c. Recuperar un archivo borrado

Para este caso, primero borramos el archivo lapapa.txt con el siguiente comando: rm /mnt/lapapa.txt

```
alumno@alumno-virtualbox: ~/Descargas/filesystem/FAT12
Archivo  Acciones  Editar  Vista  Ayuda
alumno@alumno-virtu...as/filesystem/FAT12    alumno@alumno-virtualbox: ~
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ ls /mnt
hola.txt  lapapa.txt  mi_dir  prueba.txt
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ rm /mnt/lapapa.txt
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ ls /mnt
hola.txt  mi_dir  prueba.txt
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$
```

Ejecutamos el programa *read_root* para validar que el archivo aparece como borrado. Y en la salida se observa como ?APAPA.TXT, ya que **FAT12** reemplaza el primer byte del nombre por 0xE5.

```
alumno@alumno-virtualbox: ~/Descargas/filesystem/FAT12
Archivo  Acciones  Editar  Vista  Ayuda
alumno@alumno-virtu...as/filesystem/FAT12
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ ./read_root
Encontrada particion FAT12 0
En  0x200, sector size 512, FAT size 2 sectors, 2 FATs

Root dir_entries 512
Directorio: [MI_DIR  .   ]

Archivo: [HOLA    .TXT] su contenido es:
Hola, bienvenidos !
Pueden ahora tratar de leerme desde c !


Archivo: [PRUEBA  .TXT] su contenido es:
este es un archivo de prueba

Archivo borrado: [?l.]
Archivo borrado: [?APAPA  T.TXT]
Archivo borrado: [?S.]
Archivo borrado: [?OR2TP1 T.TXT]

Leido Root directory, ahora en 0x4A00
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$
```

Luego, ejecutamos la versión modificada *read_root_recovery*.c, para recuperar el archivo borrado. El programa detecta la entrada ?APAPA .TXT, restaura el byte del nombre con la letra L y muestra el archivo recuperado como LAPAPA .TXT

```
alumno@alumno-virtualbox: ~/Descargas/filesystem/FAT12
Archivo  Acciones  Editar  Vista  Ayuda
alumno@alumno-virtu...as/filesystem/FAT12

Archivo borrado: [?l.]
Archivo borrado: [?APAPA  T.TXT]
Archivo borrado: [?S.]
Archivo borrado: [?OR2TP1 T.TXT]

Leido Root directory, ahora en 0x4A00
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ gcc read_root.c -o read_root
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ ./read_root
Encontrada particion FAT12 0
En  0x200, sector size 512, FAT size 2 sectors, 2 FATs

Root dir_entries 512
Directorio: [MI_DIR  .   ]

Archivo: [HOLA    .TXT] su contenido es:
Hola, bienvenidos !
Pueden ahora tratar de leerme desde c !


Archivo: [PRUEBA  .TXT] su contenido es:
este es un archivo de prueba

Archivo borrado: [?l.]
Archivo borrado encontrado: [?APAPA  .TXT]
Archivo recuperado como: [LAPAPA  .TXT]


Leido Root directory, ahora en 0xB00
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$
```

Finalmente, ejecutamos otra vez *read_root* y verificamos que el archivo ya no aparece como borrado.

```
alumno@alumno-virtualbox: ~/Descargas/filesystem/FAT12
Archivo  Acciones  Editar  Vista  Ayuda
alumno@alumno-virtu...as/filesystem/FAT12
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$ ./read_root
Encontrada particion FAT12 0
En  0x200, sector size 512, FAT size 2 sectors, 2 FATs

Root dir_entries 512
Directorio: [MI_DIR  .   ]

Archivo: [HOLA    .TXT] su contenido es:
Hola, bienvenidos !
Pueden ahora tratar de leerme desde c !


Archivo: [PRUEBA  .TXT] su contenido es:
este es un archivo de prueba

Archivo borrado: [?l.]
Archivo: [LAPAPA  .TXT] su contenido es:
test


Archivo borrado: [?S.]
Archivo borrado: [?OR2TP1 T.TXT]

Leido Root directory, ahora en 0x4A00
alumno@alumno-virtualbox:~/Descargas/filesystem/FAT12$
```

4. CONCLUSIONES

Se logró comprender la estructura física de FAT12, evidenciando que la eliminación de archivos no es física sino lógica (con el cambio del byte 0xE5) Esto demuestra la importancia de entender los offsets y sectores reservados para la recuperación de datos en sistemas forenses o de bajo nivel

5. REFERENCIAS

1. Design of the FAT file system. Wikipedia. https://en.wikipedia.org/wiki/Design_of_the_FAT_file_system
2. File System Data Structure. C-Jump. https://www.c-jump.com/CIS24/Slides/FileSysDataStructs/FileSysDataStructs.html

6. ANEXOS

- Repositorio de Código. GitHub. https://github.com/Ging1991/Sor2-2026-TP1

6.DECLARACIÓN DE AUTORÍA

Nombre y apellido	Firma	Legajo
1.		
2.		
3.		
4.		
5.		
6.		

Fecha: